GraphAllocBench: A Flexible Benchmark for Preference-Conditioned Multi-Objective Policy Learning

Abstract

Preference-Conditioned Policy Learning (PCPL) in Multi-Objective Reinforcement Learning (MORL) aims to approximate diverse Pareto-optimal solutions by conditioning policies on user-specified preferences over objectives. This enables a single model to flexibly adapt to arbitrary trade-offs at run-time by producing a policy on or near the Pareto front. However, existing benchmarks for PCPL algorithms are largely restricted to toy tasks and fixed environments, limiting their realism and scalability. To address this gap, we introduce GraphAllocBench, a flexible benchmark built on a novel graph-based resource allocation sandbox environment inspired by city management, which we call CityPlannerEnv. GraphAllocBench provides a rich suite of problems with diverse and customizable objective functions, varying preference conditions, complex Pareto Fronts and high-dimensional scalability. We also propose two new evaluation metrics – Proportion of Non-Dominated Solutions (PNDS) and Ordering Score (OS) – that directly capture preference consistency while complementing the widely used hypervolume metric. Through experiments with Multi-Layer Perceptrons (MLPs) and graph-aware models with our PCPL-PPO approach, we show that GraphAllocBench exposes the limitations of existing MORL approaches and paves the way for using graph-based methods such as Graph Neural Networks (GNNs) in complex, high-dimensional combinatorial allocation tasks. Beyond its predefined problem set, GraphAllocBench enables users to flexibly vary objectives, preferences, and allocation rules, establishing it as a versatile and extensible benchmark for advancing PCPL. Code: https://anonymous.4open.science/r/GraphAllocBench

1 Introduction

Reinforcement learning (RL) has achieved remarkable progress in domains such as video games [Shao *et al.*, 2019], robotics [Labiosa *et al.*, 2025; Luo *et al.*, 2025], and large language model fine-tuning [Ouyang *et al.*, 2022]. Much of this success is built on single-objective RL algorithms such as Proximal Policy Optimization (PPO) [Schulman *et al.*, 2017], which optimize a single scalar reward function.

However, many real-world and simulated environments involve multiple, often conflicting objectives rather than a single scalar reward. For example, in games and training simulations, Non-Player Characters (NPCs) must balance dealing damage, avoiding damage, and maximizing survival time [Schrum and Miikkulainen, 2008] to create engaging and adaptive experiences. In such cases, objectives carry different levels of importance; we term such weights *preferences* over objectives.

A common approach to multi-objective RL (MORL) is scalarization [Van Moffaert *et al.*, 2013] (See Appendix D.3) , where objectives are collapsed into a single reward using preference weights. While straightforward, using a predetermined scalarization ties the learned policy to one fixed preference and requires retraining for new ones.

Other existing approaches to MORL (See Section 2.1) include training populations of policies via evolutionary algorithms [Deb and Jain, 2014; Dixit and Tumer, 2025], particle swarm approaches [Becerril Torres *et al.*, 2025] or training multiple single-objective models to approximate the Pareto front [Xu *et al.*, 2020]. These methods are generally inefficient, as each new preference requires retraining from scratch, or requires keeping track of multiple concurrent policies. Gradient-based optimization with vector-valued rewards [Désidéri, 2012] has also been explored, but often suffers from convergence issues [Liu *et al.*, 2021].

Preference-conditioned Policy Learning (PCPL) in MORL, for instance PD-MORL [Basaklar *et al.*, 2023], offers a more scalable solution. Instead of training a separate model for each preference, a single preference-conditioned policy can adapt to arbitrary user-specified preferences. Compared to alternatives such as role embeddings [Long *et al.*, 2024], policy pools [Garnelo *et al.*, 2021], or mixture-of-experts models [Martinez-Gil and Gil-Magraner, 2025], preference-conditioned methods provide a more direct and flexible mechanism for handling diverse objectives.

Despite algorithmic progress in the field, current multi-objective benchmarks [Felten *et al.*, 2023] are insufficient for pushing progress forward for PCPL. Most existing testbeds utilize either continuous optimization problems

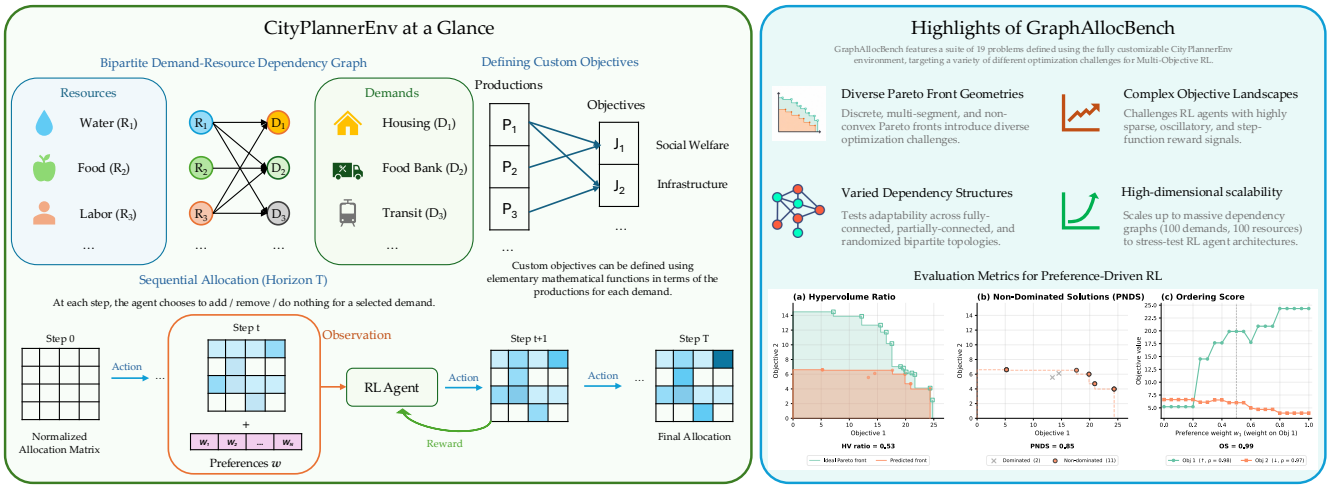

Figure 1: CityPlannerEnv models city resource allocation as sequential decision-making over a bipartite demand-resource graph, with composable multi-objective rewards and preference-conditioned policies. GraphAllocBench provides a diverse suite of problems spanning varied graph structures, Pareto front geometries, and scales, built on top of CityPlannerEnv.

[Deb *et al.*, 2005; Huband *et al.*, 2006] or simple 2D grid-based games [Yang *et al.*, 2019; Vamplew *et al.*, 2011]. These lack flexibility in scaling observations, actions, objectives and dependency structures, making them insufficient to evaluate the robustness of preference-conditioned methods in more complex problems. For instance, in Deep Sea Treasure [Vamplew *et al.*, 2011], the objectives are to maximize treasure and minimize time, as limited by the game mechanics.

To address this gap, we introduce **CityPlannerEnv**, a Gymnasium-based [Towers *et al.*, 2024] sandbox environment inspired by city-scale resource allocation that enables multi-objective planning in graph-based environments of arbitrary and scalable complexity. Cities must allocate limited resources across diverse and often conflicting demands – such as reducing congestion, fostering economic growth, and promoting sustainability – according to shifting preferences. The environment models this process as a sequential allocation problem, represented through bipartite resource-demand dependency graphs. Agents must not only balance competing objectives but also anticipate future events and adapt to varying risk profiles (e.g., conserving resources for emergencies versus aggressively pursuing short-term gains).

In **CityPlannerEnv**, agents allocate resources by adding or removing productions for each demand at each step to pursue custom-defined objectives based on user preferences. This incremental allocation mechanism draws inspiration from the Multi-step Colonel Blotto Game (MCBG), a well-studied model for competitive resource allocation on graphs [An and Zhou, 2025b; An and Zhou, 2025a; Shishika *et al.*, 2022]. Dependencies between resources and demands are represented as bipartite graphs, and objectives can be defined over any subset of productions. Agents seek to plan actions that maximize the expected cumulative reward over time while respecting preference trade-offs. As the number of resources, demands, and objectives increases – and as conditions change over time – the allocation problem becomes increasingly challenging for PCPL.

We further introduce **GraphAllocBench**, a benchmark for PCPL that defines a suite of challenging problems designed using CityPlannerEnv. GraphAllocBench leverages the flexibility of CityPlannerEnv to create customizable problems by varying the numbers of demands, resources, objectives, and objective shapes, providing targeted stress tests for PCPL algorithms. For example, GraphAllocBench includes test problems spanning diverse optimization challenges, including difficult objective functions, diverse and non-convex Pareto fronts, sparse observation spaces, complex dependency structures, and high-dimensional graph observations. For performance characterization, the benchmark includes the hypervolume ratio along with two novel supplementary metrics – Proportion of Non-Dominated Solutions (PNDS) and Ordering Score (OS) – to assess how well policies satisfy varied preferences. Figure 1 shows an overview of CityPlannerEnv and GraphAllocBench.

To establish baseline performance and highlight how the benchmark challenges existing methods, we demonstrate a PPO-based [Schulman *et al.*, 2017] PCPL solution (PCPL-PPO) with Multi-Layer Perceptron (MLP) and Heterogeneous Graph Neural Network (HGNN) [Hu *et al.*, 2020] feature extractors. We further employ Smooth Tchebycheff Scalarization [Lin *et al.*, 2024] to approximate challenging non-convex Pareto fronts and demonstrate how graph neural networks can learn preference-conditioned, graph-aware policies in environments with complex dependency structures, going beyond prior work that primarily focused on 2D grids [Basaklar *et al.*, 2023] or simpler graph topologies [Lin *et al.*, 2022; Fu and Gu, 2024].

Our contributions are as follows:

- **GraphAllocBench benchmark:** We introduce *GraphAllocBench*, a flexible and challenging graph-

based benchmark for preference-conditioned policy learning, based on our novel, customizable sandbox environment *CityPlannerEnv*, designed to capture complex resource allocation scenarios with complex bipartite dependency structures, discrete non-convex Pareto Fronts, and difficult objective functions.

- **Novel evaluation metrics:** Beyond the standard hypervolume metric, we introduce the *ordering score* (OS) and the *proportion of non-dominated solutions* (PNDS) to evaluate preference-consistency and robustness, addressing limitations of existing multi-objective evaluation practices for PCPL algorithms (see Section 2).
- **PCPL-PPO with MLPs and HGNNs:** We use MLP-based PCPL-PPO as a strong baseline, and raise the performance ceiling with preference-conditioned HGNN-based PCPL-PPO, showing improved generalization and preference-awareness on high-dimensional graph observations.

2 Background

2.1 Multi-Objective RL

Many solutions in the multi-objective optimization space have aimed to tackle the issue of learning Pareto Fronts (See Appendix A). Evolutionary methods such as NSGA-III [Deb and Jain, 2014] have shown success, focusing on generating increasingly Pareto-optimal populations of solutions with each generation by performing operations such as selection, crossover and mutation. However, while these evolutionary methods are effective for simpler multi-objective optimization tasks, they are less adaptable towards more complex and high-dimensional tasks, such as graphs, and struggle to maintain a set of diverse solutions [Gu *et al.*, 2022].

More importantly, popular multi-objective optimization algorithms including evolutionary methods often aim to produce a set of solutions on the Pareto Front, making it difficult to specify a preference for a particular solution directly as input. Learning and maintaining a large population of Pareto-optimal solutions can be computationally and memory intensive, especially for higher-dimensional objective spaces. Hence for a large number of objectives, it becomes difficult to maintain a good resolution of different solutions on the Pareto Front.

For Pareto-Optimal Policy Set Learning in RL, Multiple-Gradient Descent Algorithms (MGDA) [Désidéri, 2012] have been introduced to approximate the Pareto Front. These algorithms involve iteratively updating variables such that all objectives are improved at the same time. However, these methods do not allow us to easily specify preferences for solutions with different tradeoffs, and face convergence issues for some segments on the Pareto Front [Liu *et al.*, 2021].

Some work has been conducted to learn preference-aware Pareto-optimal policies using RL. One such approach is to train multiple single-objective RL models on a set of fixed preference vectors, and select a corresponding model to use during inference, based on a specified preference. A variant of this method uses an evolutionary algorithm with a population of different RL policies [Xu *et al.*, 2020]. A limitation of this method is that we can only learn a discrete set of preferences, as we need to train a separate RL model for each fixed preference vector. Additionally, these methods struggle with similar challenges to traditional multi-objective optimization tasks, as they still involve generating a population of Pareto-optimal solutions instead of an individual preference-aware solution.

Unlike conventional multi-objective approaches that generate a complete Pareto Front, Preference-Conditioned Policy Learning (PCPL) methods [Basaklar *et al.*, 2023; Lin *et al.*, 2022; Liu *et al.*, 2025; Yang *et al.*, 2019] use a continuous, user-defined preference as input to a single policy. This allows them to produce a specific, preference-aware solution on the Pareto Front via a series of sequential decisions within the environment. The final reward can be computed using the scalarization (See Appendix D.3) of the components of the objective function, based on the preference provided during input. This allows us to learn a preference-conditioned policy, where during inference, we can specify any preference vector within a continuous range and obtain an approximation of the Pareto-optimal solution on the Pareto Front.

2.2 Benchmarking MORL

Several benchmarks have been proposed for multi-objective optimization and reinforcement learning. For multi-objective optimization problems, examples include DTLZ [Deb *et al.*, 2005] and WFG [Huband *et al.*, 2006], which are primarily traditional multi-objective optimization problems, where we are varying decision variables to approximate mostly continuous objective functions and Pareto Fronts. These benchmarks offer expandable and flexible testcases for multi-objective optimization. However, these benchmarks are not easily adaptable for MORL tasks involving sequential decision making, non-continuous Pareto Fronts, or high dimensional observation spaces such as graphs.

Common benchmarks for MORL include game environments, for instance those in Multi-Objective Gymnasium [Felten *et al.*, 2023] such as Deep Sea Treasure (DST) [Vamplew *et al.*, 2011] and Fruit Tree Navigation (FTN) [Yang *et al.*, 2019]. These are game environments, e.g. where an agent tries to accomplish multiple objectives in a 2D grid world or MuJoCo task with continuous action spaces [Zhu *et al.*, 2023]. However, these benchmarks are still limited to the mechanics of the game environment itself (limited action and observation spaces), and it is difficult to design custom scenarios and objective functions to challenge the RL model. Even though variants of these problems [Cassimon *et al.*, 2021] have also been introduced to increase the size of the observation space and number of objectives, these variants are still non-expandable and normally focus on small action and observation spaces, with limited number of objectives, often just 2-3 objectives [Zhu *et al.*, 2023].

GraphAllocBench addresses these gaps along three axes that existing benchmarks lack: (1) customizable objectives, enabling deliberate design of Pareto Front geometry; (2) scalable bipartite graph structure, exposing the limits of MLP-based approaches; and (3) complementary evaluation metrics (PNDS and OS) that go beyond hypervolume: PNDS measures prediction reliability, while OS is a preference-specific

metric that measures preference alignment.

3 CityPlannerEnv

Our sandbox environment **CityPlannerEnv** is modeled as a city that has a set of resources R and a set of demands D (Figure 1). At each step, the RL agent receives the current resource allocation state as an observation, takes an action to alter the allocation, and receives a multi-objective reward signal.

3.1 Definitions

Resource This refers to a particular factor of production, denoted by R_i, which can be combined with other resources to create productions (e.g. water, food, labor). The environment starts with a predefined number of units for each resource.

Demand A type of need fulfilled by resources (e.g. food bank), denoted by D_i.

Requirements Unweighted dependency set $R'(D_i) \subseteq R$ of the resources needed by each demand D_i.

Allocation Assignment of resources to demands, represented by allocation matrix A, where $A_{i,j}$ is the amount of R_i given to D_j. The observation returned by the environment is the normalized allocation matrix, concatenated with the current preferences vector for each rollout.

Production Actual output for each demand, determined by the limiting allocated resources. Production $\mathbf{P}$ changes when an action is taken to add or remove a production, where $|\mathbf{P}|$ is the number of demands.

Objective Function Multi-objective reward signal at the end of each step (i.e. after each action), defined by $\mathbf{J}(\mathbf{P}) : \mathbb{R}^{|\mathbf{P}|} \to \mathbb{R}^N$, mapping $|\mathbf{P}| = |D|$ productions to N objective values.

Preferences Importance assigned to each objective, represented by preferences vector $\mathbf{w} = [w_1, \ldots, w_N]$, $\sum_{n=1}^{N} w_n = 1$, $0 \leq w_i \leq 1$ for each $i \in \{1, \ldots, N\}$.

3.2 Action Space

Custom action spaces can be defined within the CityPlannerEnv environment to move resources between demands. In our solution, we define a single action as the selection of 2 sub-actions. Firstly, the agent will choose to either add or remove a production or not to modify the current allocation state. Secondly, the agent will choose a specific demand among the demands D for which to create 1 unit of production, remove 1 unit of production, or do nothing, depending on the first action chosen. When we add or remove a production, the corresponding resources tied to that demand will also be removed or added respectively from the unallocated resources pile.

CityPlannerEnv is implemented using Gymnasium [Towers *et al.*, 2024]. Custom environment configurations can be designed for the number of resource and demand types, the number of available resources for each type of resource, resource-demand dependency graphs, and objective functions using common mathematical functions (e.g. polynomial, sinusoidal, logarithmic).

3.3 Custom Objectives

Objective functions in CityPlannerEnv are fully user-definable: each component J_i can depend on any subset of productions P_j and can be composed from elementary functions (linear, quadratic, logarithmic, logistic, Gaussian, ceiling, and floor, or sums thereof). This composability allows practitioners to deliberately engineer Pareto Front geometries, including smooth convex fronts, discontinuous multi-segment fronts, and non-convex fronts with oscillatory structure. Table 3 in Appendix E illustrates representative objective functions spanning these cases. This design flexibility is a key differentiator from existing benchmarks, where objectives are fixed by game mechanics.

4 GraphAllocBench

GraphAllocBench comprises a set of 19 challenging evaluation problems designed using our sandbox environment CityPlannerEnv, across six challenge categories: Problem 0 serves as a baseline with a smooth, convex Pareto front; Problems 1a–1c introduce oscillatory and multi-segment objective landscapes with sparse rewards; Problems 2a–2c increase the number of demands, expanding the action and observation spaces; Problems 3a–3b scale to five objectives with highly sparse reward signals; Problems 4a–4b use non-fully-connected dependency graphs; and Problems 5a–5e feature randomly sampled dependency structures and objective functions at extended planning horizons. Three large-scale problems (6a–6c) further stress-test scalability on graphs with 100 demands. Full problem specifications are provided in Appendix E (Table 2).

This benchmark evaluates the quality of solutions using the hypervolume (HV) and Proportion of Non-Dominated Solutions (PNDS) using preferences sampled with the Das and Dennis method [Das and Dennis, 1998]. The solutions evaluated via the Ordering Score (OS) are sampled separately (See Algorithm 1 in Appendix C).

For each sampled preference, we roll out the agent's policy deterministically for T steps to obtain the final allocation of resources to demands. The Das and Dennis method [Das and Dennis, 1998] generates a set of evenly distributed preferences by constructing a structured grid of points on an N-dimensional simplex (for N objectives). Hence, the number of preferences sampled increases combinatorially with N. For scenarios where it is computationally feasible to compute the allocations that lead to solutions on the analytical Pareto Front, we compare the hypervolume of the set of predicted solutions for the sampled preferences with the ideal Pareto Front. The evaluation metrics are defined respectively in the following sections.

4.1 Hypervolume

For each sampled preference $\mathbf{w}$, we obtain a final allocation of resources, which allows us to calculate the final production vector $\mathbf{P}$ for a given allocation. Hence, we can use the

final production state to calculate the value of each objective component, which are collectively represented by $\mathbf{J}(\mathbf{P})$. By varying our preferences, we obtain sets of objective vectors $\mathbf{J}$, which form a set of predicted optimal solutions (i.e. the predicted Pareto Front). We can evaluate this solution set using the hypervolume metric based on the maximization variant of the PyMOO [Blank and Deb, 2020] implementation (See Appendix B).

For simpler solutions, we can also calculate the ideal hypervolume of the environmental setup by enumerating all possible allocations. We can then calculate how well the predicted Pareto Front estimates the true Pareto Front using $\text{HV}_{\text{ratio}} = \text{HV}_{\text{predicted}}/\text{HV}_{\text{ideal}}$.

By calculating the ratio of the measured hypervolume to the ideal analytical hypervolume, we are able to measure the performance of the trained RL agent against the best possible solution. We chose this metric over other metrics such as the Generational Distance (GD), which measures the average distance of objective vectors from the Pareto Front, because the hypervolume is more robust towards discontinuous Pareto Fronts, whereas other metrics are more suited to continuous Pareto Fronts. In addition, by measuring the ratio of the measured hypervolume over the ideal hypervolume, we can compare normalized scores across problems with different number of objectives more easily, as raw hypervolume values are not comparable across different objective dimensions. For scenarios where the ideal hypervolume is not available due to computational complexity (high number of demands or objectives), we use the raw hypervolume to compare across different algorithms or architectures for each fixed task.

While hypervolume-based metrics can evaluate the overall quality and diversity of solutions, measuring the hypervolume becomes computationally expensive for high numbers of objectives, and may not provide the full picture in evaluating multi-objective problems [Ibrahim *et al.*, 2024], especially for PCPL. In particular, an agent may predict solutions close to the ideal Pareto Front some of the time, resulting in a high hypervolume, but the hypervolume metric does not indicate the reliability of single-shot predictions.

4.2 Proportion of Non-Dominated Solutions (PNDS)

To address the issue of prediction reliability, we also calculate the proportion of the predicted solutions from Section 4.1 that are non-dominated. This metric is independent of the ideal Pareto Front solution, which provides two main advantages:

1. We are able to evaluate this metric even if it is computationally infeasible to evaluate the ideal Pareto Front (e.g. high number of production combinations).
2. If the proportion of non-dominated solutions is high but the hypervolume is low, this can be an indication that our agent is trapped in local optima.

While this metric is beneficial as a supplement to the hypervolume, it still struggles when the number of objectives is high due to sampling inefficiency and requires sampling exponentially more preferences using [Das and Dennis, 1998] with increasing N to ensure fair comparisons.

4.3 Ordering Score (OS)

The ordering score measures the rank correlation between the preference weight w_i on each production i and the value of the objective function $\mathbf{J}_i(\mathbf{P})$. This evaluates how well the agent obeys the preference input, instead of whether the agent generates solutions close to the Pareto Front, which is measured by the hypervolume and PNDS. For traditional multi-objective optimization algorithms, we are often concerned with generating an unordered population of solutions which approximates the Pareto Front. However, for PCPL, the ordering score becomes an important way to evaluate how well an agent follows the preference it is given.

To compute the ordering score detailed in Algorithm 1 in Appendix C, we sample preferences that sweep along each objective and evaluate the corresponding objective values. An agent has performed well if the reward received in a particular objective increases with the preference weight in that component. We quantify this relationship using the Spearman Rank Correlation, normalized between 0 and 1 and averaged across all objective dimensions. The final ordering score is computed as the average Spearman Rank Correlation across all samples and all objectives.

While the ordering score is attractive as a low-cost, scalable metric for high-dimensional objectives, it does not account for the changes in magnitude of the objective values for each preference dimension, and only rewards the discrete preference-alignment of the achieved objective values. However, this is also an advantage, as the Spearman Rank Correlation does not make any distributional assumptions, rendering it robust towards outliers and a good complement for the HV metric, which focuses on the magnitudes of the objective values. Additionally, the OS is a unique metric specifically intended for PCPL problems, whereas HV and PNDS do not account for preference alignment, a factor which distinguishes PCPL problems from regular multi-objective optimization or RL. We therefore view the ordering score as a supplementary diagnostic for preference-awareness, used alongside the hypervolume and PNDS to present a more complete evaluation picture.

5 Results and Discussion

5.1 Main Problem Set

Using the set of objective functions as our multi-objective reward function $\mathbf{J}$, we trained a preference-conditioned policy using Proximal Policy Optimization (PPO) on GraphAllocBench to convergence (See Figure 1). We will refer to our implementation as PCPL-PPO. We then employ the methods in Section 4 to evaluate the trained model. During training, preferences are sampled from a flat Dirichlet distribution for each rollout, and the PPO agent is trained to maximize the scalarized reward (using Smooth Tchebycheff Scalarization [Lin *et al.*, 2024]), given the sampled preference vector and current normalized allocation matrix as input. Each rollout lasts for a total of T steps. See Appendix D for more information on our PCPL-PPO implementation. We evaluate the performance of PCPL-PPO policies on GraphAllocBench for problem categories 0-5, and compare against PD-MORL [Basaklar *et al.*, 2023], a state-of-

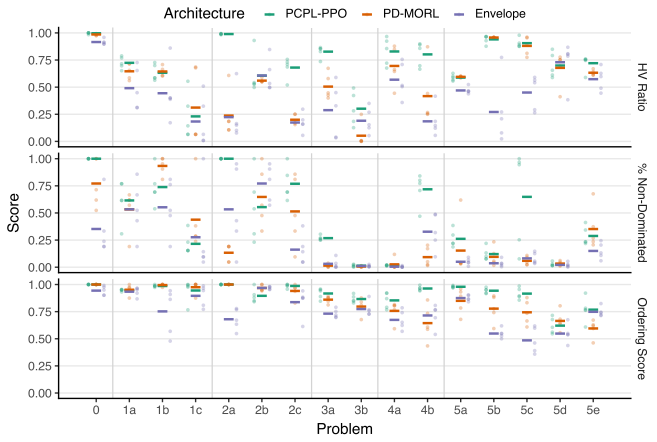

Figure 2: Performance comparison of PCPL-PPO, PD-MORL and Envelope Q-Learning policies over 5 random seeds at 1M steps for HV Ratio, PNDS and OS. Horizontal bars show the mean, and dots show the outcome for each seed.

the-art preference-conditioned MORL algorithm, as the primary baseline. Figure 2 summarizes the performance across the problems 0-5 for PCPL-PPO (our method with full hyperparameter sweep), PD-MORL [Basaklar *et al.*, 2023] and Envelope Q-Learning [Yang *et al.*, 2019]. Across different methods, the benchmark's diverse problem set elicits distinct failure modes – no single method dominates across all metrics – and the supplementary PNDS and OS metrics reveal preference-alignment gaps that hypervolume alone does not capture. Full numerical results are available in Appendix F.

Sharp Changes in Rewards (e.g. Problems 1a-c)

These sudden changes in reward signals make it difficult for an agent to learn a smooth approximation of the Pareto Front, and also lead to greater variance in predictions. From Figure 2, compared to the baseline (Problem 0) with a smooth objective function, Problems 1a-c report significantly higher variance and worse approximations of the Pareto Front despite sharing a dependency structure with the baseline, since sharp objective functions can result in large jumps between adjacent discrete points on the Pareto Front, which can be difficult to predict based on a smooth preference vector input.

Sparse Rewards (e.g. Problem 1c)

A PCPL agent may find it difficult to deal with sparse rewards, especially when optimizing a continuous preference vector. As example of this would be the $\mathbf{J}_i(\mathbf{P}) = \max(0, \mathbf{P}_j - 5)$ floor function, where we receive no reward for Objective i until we produce more than 5 units for Demand j. Instead of deciding whether to a) optimize a particular objective by creating productions until the reward spikes or to b) allocate no resources to the given production, the agent might create insufficient production, such that resources are wasted but no additional reward is received. This results in a lower proportion of non-dominated solutions, as observed for Problem 1c in Figure 2. In Figure 3, we also observe many predicted solutions along the horizontal axis where the reward component for Objective 1 is 0. However, there should only be one Pareto-optimal solution along the horizontal axis that maximizes Objective 0.

Non-convex Pareto Front (e.g. Problems 1c, 3b)

Non-convex Pareto Fronts are difficult to represent in general for multi-objective optimization or RL problems, and are highly dependent on the scalarization function used. Smooth Tchebycheff Scalarization is more robust to non-convexity than weighted sums, but still has limitations when the Pareto front is non-convex and discontinuous, as shown in Problem 1c and 3b (See Figures 2 and 3).

Unbalanced Objectives (e.g. Problem 2c)

Objectives with sparse reward signal can be difficult to optimize jointly with other concurrent objectives, which can result in failure to reach the sparser objective's extreme values and achieving a lower-quality approximation of intermediate values on the Pareto Front (See Problem 2c in Figure 2).

Local Optima Traps (e.g. Problem 2b)

When approximating a continuous Pareto front from smooth preference inputs, the PCPL agent can fail to capture certain regions of the front in the presence of local optima (Problem 2b). As shown in Figure 3, the agent collapses to local solutions and cannot reliably generalize to the global Pareto front, highlighting a key failure mode of existing approaches.

6 Heterogeneous Graph Neural Network

For problems 6a-c with a large number of resources and productions, we used a Heterogeneous Graph Neural Network (HGNN) [Hu *et al.*, 2020] to represent the complex bipartite structure of the demand-resource dependencies. Extracting global information from Graph Neural Networks to use as feature input to RL models is challenging, because global pooling methods can potentially dilute the quantity and quality of information available in the original environment. To tackle this issue, past works have used specific node embeddings [Wang *et al.*, 2025] and node embedding concatenation [Sharma and Kunkel, 2025]; thus, we hope to show that flexible pooling methods like Mean/Max Pooling and Attention Pooling can still capture global information more effectively than MLPs. We replace the common MLP feature extractor with a HGNN-based feature extractor by using multiple Heterogeneous Graph Attention layers and performing global pooling on the node embeddings for different types of nodes separately. Each HGNN layer is comprised of several Graph Attention Networks [Veličković *et al.*, 2018], one for each type of node (Demand / Resource / Unallocated). Two HGNN layers are stacked with residual connections in between.

6.1 HGNN Preference Conditioning

To increase the preference-awareness of the policy, we incorporate preference conditioning at multiple stages within the HGNN feature extractor and training pipeline, enhancing the model's robustness to varying preference inputs.

Concatenating Preferences to Features

We concatenate the preference vector to both the initial node embeddings and the final pooled node embeddings. Additionally, residual connections between HGNN layers are

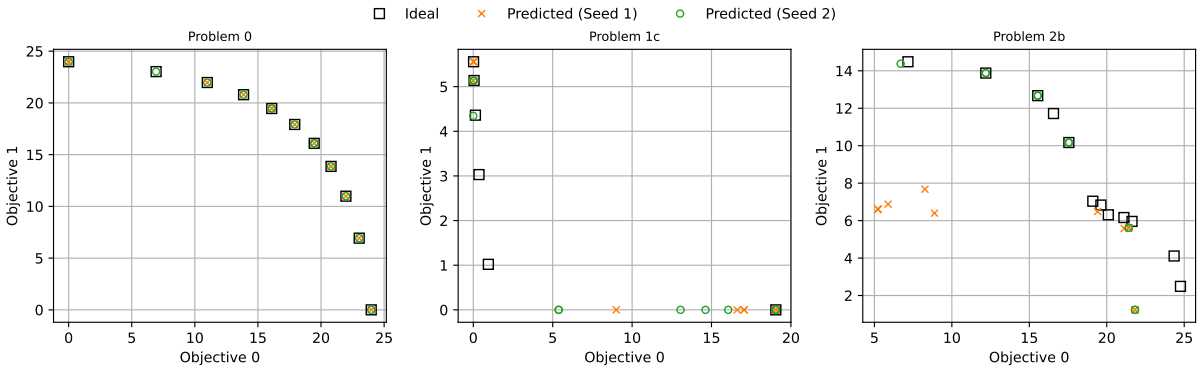

Figure 3: Selected 2-Objective Pareto Fronts for PCPL-PPO at 1M steps over 2 random seeds: Compared to the Problem 0 baseline, the PCPL-PPO agent struggles with non-convex (Problem 1c) and multi-segment (Problem 2b) Pareto Fronts.

Table 1: Performance comparison of PCPL-PPO with MLP (1.58M params), HGNN+MeanMaxPool (635K params), and HGNN+AttentionPool (1.15M params) feature extractors at 1M steps across 5 random seeds. The hypervolume (HV) measures overall policy performance (mean $\pm$ std). The mean scores for PNDS and Ordering Score measure consistency and preference alignment.

Problem 6a	HV $\times 10^6$	PNDS	Order
MLP	3.3 ± 0.7	0.06	**0.88**
HGNN+MeanMaxPool	6.7 ± 2.5	**0.15**	0.75
HGNN+AttentionPool	$\mathbf{12.9 \pm 1.5}$	0.07	0.68
Problem 6b	HV $\times 10^6$	PNDS	Order
MLP	3.1 ± 0.6	0.06	**0.85**
HGNN+MeanMaxPool	6.8 ± 2.2	**0.13**	0.73
HGNN+AttentionPool	$\mathbf{12.7 \pm 1.2}$	0.08	0.67
Problem 6c	HV $\times 10^6$	PNDS	Order
MLP	5.5 ± 0.9	0.06	**0.84**
HGNN+MeanMaxPool	18.4 ± 2.1	**0.14**	0.59
HGNN+AttentionPool	$\mathbf{30.2 \pm 2.4}$	0.05	0.65

employed to preserve preference information across layers throughout the forward pass.

Preference-conditioned Attention Pooling

We use preference-conditioned multi-head attention pooling, where each head computes attention scores over the nodes by jointly considering the node features and the preference vector $\mathbf{w}$. For each head, the node embeddings are transformed together with $\mathbf{w}$, weighted by the attention scores, and aggregated into a pooled representation. The outputs of all heads are then concatenated to form the final preference-aware global graph representation.

6.2 MLP vs HGNN Model Comparison

For small graphs, MLP-based models are preferred for their stability and rate of convergence. However, Graph Neural Networks have the potential to better generalize to large graphs with large dependency structures and to eventually outperform MLPs. We investigated this behavior for Problems 6a-c in our evaluation problem set by increasing the number of demands and resources to 100 each, with a total of 201 nodes in the graph (including the node for unallocated resources). From Table 1, we can see that the HGNN-based approaches achieve significantly and consistently better hypervolume scores compared to the MLP-based counterpart. Preference-conditioned multi-headed attention pooling also led to improved hypervolume, as compared to a simple concatenation of mean and max pooling. Both HGNN methods have fewer parameters than the MLP-based PPO model, as the number of MLP parameters scales quadratically with the size of the allocation matrix, due to the flattening of the allocation matrix and the initial fully connected feedforward neural network. The MLP outperforms the HGNN versions on ordering score, which may indicate that the MLP finds more stable and better preference-conditioned local solutions than the HGNN-based models, but struggles to find solutions closer to the global solution (See Section 4).

7 Conclusion

In this paper, we introduced **GraphAllocBench**, a flexible benchmark for preference-aware RL and combinatorial resource allocation powered by our custom **CityPlannerEnv** sandbox. GraphAllocBench's diverse problem set exposes distinct failure modes across both SOTA methods and our proposed PCPL-PPO approach, demonstrating that no single method dominates across all settings. Crucially, our supplementary metrics PNDS and Ordering Score reveal preference-alignment gaps that hypervolume alone cannot capture: a solution with lower HV may better reflect user preferences as measured by OS, underscoring the importance of multi-dimensional evaluation for PCPL. Our experiments further show that equipping PPO with a Heterogeneous Graph Neural Network (HGNN) feature extractor outperforms standard MLP-based baselines on large graphs, though trade-offs in preference consistency remain an open challenge. Overall, GraphAllocBench serves as a versatile and challenging testbed for advancing research in preference-aware many-objective policy learning, and future work will extend the benchmark with richer dependency structures and environmental uncertainty to evaluate risk-aware decision-making.

References

[An and Zhou, 2025a] Zijian An and Lifeng Zhou. Double oracle algorithm for game-theoretic robot allocation on graphs. *IEEE Transactions on Robotics*, 41:3244–3259, 2025.

[An and Zhou, 2025b] Zijian An and Lifeng Zhou. Reinforcement learning for game-theoretic resource allocation on graphs. *IEEE transactions on automation science and engineering*, 2025.

[Basaklar *et al.*, 2023] Toygun Basaklar, Suat Gumussoy, and Umit Ogras. PD-MORL: Preference-driven multi-objective reinforcement learning algorithm. In *The Eleventh International Conference on Learning Representations*, 2023.

[Becerril Torres *et al.*, 2025] Teresa Becerril Torres, Carlos Ignacio Hernández Castellanos, and Roberto Santana Hermida. Particle swarm algorithm for multi-objective reinforcement learning. In *Proceedings of the 4th Workshop on Multi-Objective Decision Making (MODeM 2025)*, Bologna, Italy, 2025. https://modem2025.vub.ac.be/papers/MODeM_2025_paper_8.pdf.

[Blank and Deb, 2020] J. Blank and K. Deb. pymoo: Multi-objective optimization in python. *IEEE Access*, 8:89497–89509, 2020.

[Cassimon *et al.*, 2021] Amber Cassimon, Reinout Eyckerman, Siegfried Mercelis, Steven Latré, and Peter Hellinckx. A review of the deep sea treasure problem as a multi-objective reinforcement learning benchmark. *CoRR*, abs/2110.06742, 2021.

[Das and Dennis, 1998] Indraneel Das and J. E. Dennis. Normal-boundary intersection: A new method for generating the pareto surface in nonlinear multicriteria optimization problems. *SIAM Journal on Optimization*, 8(3):631–657, 1998.

[Deb and Jain, 2014] Kalyanmoy Deb and Himanshu Jain. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part i: Solving problems with box constraints. *IEEE Transactions on Evolutionary Computation*, 18(4):577–601, 2014.

[Deb *et al.*, 2005] Kalyanmoy Deb, Lothar Thiele, Marco Laumanns, and Eckart Zitzler. *Scalable Test Problems for Evolutionary Multiobjective Optimization*, pages 105–145. Springer London, London, 2005.

[Dixit and Tumer, 2025] Gaurav Dixit and Kagan Tumer. Preference-conditioned evolutionary reinforcement learning for multi-objective continuous control. In *Proceedings of the 4th Workshop on Multi-Objective Decision Making (MODeM 2025)*, Bologna, Italy, 2025. https://modem2025.vub.ac.be/papers/MODeM_2025_paper_14.pdf.

[Désidéri, 2012] Jean-Antoine Désidéri. Multiple-gradient descent algorithm (mgda) for multiobjective optimization. *Comptes Rendus Mathematique*, 350(5):313–318, 2012.

[Felten *et al.*, 2023] Florian Felten, Lucas N. Alegre, Ann Nowé, Ana L. C. Bazzan, El Ghazali Talbi, Grégoire Danoy, and Bruno C. da Silva. A toolkit for reliable benchmarking and research in multi-objective reinforcement learning. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023)*, 2023.

[Fu and Gu, 2024] Xiaoyu Fu and Shenshen Gu. Deep reinforcement learning algorithm based on graph weight multi-pointer network for solving multiobjective traveling salesman problem. *IEEE Access*, 12:179091–179103, 2024.

[Garnelo *et al.*, 2021] Marta Garnelo, Wojciech Marian Czarnecki, Siqi Liu, Dhruva Tirumala, Junhyuk Oh, Gauthier Gidel, Hado van Hasselt, and David Balduzzi. Pick your battles: Interaction graphs as population-level objectives for strategic diversity. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '21, page 1501–1503, Richland, SC, 2021. International Foundation for Autonomous Agents and Multiagent Systems.

[Gu *et al.*, 2022] Qinghua Gu, Qingsong Xu, and Xuexian Li. An improved nsga-iii algorithm based on distance dominance relation for many-objective optimization. *Expert Systems with Applications*, 207:117738, 2022.

[Hu *et al.*, 2020] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. *CoRR*, abs/2003.01332, 2020.

[Huband *et al.*, 2006] S. Huband, P. Hingston, L. Barone, and L. While. A review of multiobjective test problems and a scalable test problem toolkit. *IEEE Transactions on Evolutionary Computation*, 10(5):477–506, 2006.

[Ibrahim *et al.*, 2024] Amin Ibrahim, Azam Asilian Bidgoli, Shahryar Rahnamayan, and Kalyanmoy Deb. A novel pareto-optimal ranking method for comparing multi-objective optimization algorithms, 2024.

[Ishibuchi *et al.*, 2016] Hisao Ishibuchi, Ryo Imada, Yu Setoguchi, and Yusuke Nojima. Performance comparison of nsga-ii and nsga-iii on various many-objective test problems. *2016 IEEE Congress on Evolutionary Computation (CEC)*, pages 3045–3052, 2016.

[Labiosa *et al.*, 2025] Adam Labiosa, Zhihan Wang, Siddhant Agarwal, William Cong, Geethika Hemkumar, Abhinav Narayan Harish, Benjamin Hong, Josh Kelle, Chen Li, Yuhao Li, Zisen Shao, Peter Stone, and Josiah P. Hanna. Reinforcement learning within the classical robotics stack: A case study in robot soccer, 2025.

[Lin *et al.*, 2022] Xi Lin, Zhiyuan Yang, and Qingfu Zhang. Pareto set learning for neural multi-objective combinatorial optimization. In *International Conference on Learning Representations*, 2022.

[Lin *et al.*, 2024] Xi Lin, Xiaoyuan Zhang, Zhiyuan Yang, Fei Liu, Zhenkun Wang, and Qingfu Zhang. Smooth tchebycheff scalarization for multi-objective optimization. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

[Liu *et al.*, 2021] Xingchao Liu, Xin Tong, and Qiang Liu. Profiling pareto front with multi-objective stein variational gradient descent. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 14721–14733. Curran Associates, Inc., 2021.

[Liu *et al.*, 2025] Erlong Liu, Yu-Chang Wu, Xiaobin Huang, Chengrui Gao, Ren-Jian Wang, Ke Xue, and Chao Qian. Pareto set learning for multi-objective reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 18789–18797, 2025.

[Long *et al.*, 2024] Weifan Long, Wen Wen, Peng Zhai, and Lihua Zhang. Role play: Learning adaptive role-specific strategies in multi-agent interactions, 2024.

[Luo *et al.*, 2025] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105):eads5033, 2025.

[Martinez-Gil and Gil-Magraner, 2025] Francisco Martinez-Gil and Eduard Gil-Magraner. Multi-agent reinforcement learning for creating intelligent agents in social networks-oriented role playing games. *Entertainment Computing*, 54:100941, 2025.

[Ouyang *et al.*, 2022] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc., 2022.

[Raffin *et al.*, 2021] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[Schrum and Miikkulainen, 2008] Jacob Schrum and Risto Miikkulainen. Constructing complex npc behavior via multi-objective neuroevolution. In *Proceedings of the Fourth Artificial Intelligence and Interactive Digital Entertainment Conference (AIIDE 2008)*, pages 108–113, Stanford, California, 2008.

[Schulman *et al.*, 2017] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *ArXiv*, abs/1707.06347, 2017.

[Shao *et al.*, 2019] Kun Shao, Zhentao Tang, Yuanheng Zhu, Nannan Li, and Dongbin Zhao. A survey of deep reinforcement learning in video games. *CoRR*, abs/1912.10944, 2019.

[Sharma and Kunkel, 2025] Aasish Kumar Sharma and Julian Kunkel. Grapheon rl: A graph neural network and reinforcement learning framework for constraint and data-aware workflow mapping and scheduling in heterogeneous hpc systems. In *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, pages 489–494, 2025.

[Shishika *et al.*, 2022] Daigo Shishika, Yue Guan, Michael Dorothy, and Vijay Kumar. Dynamic defender-attacker blotto game. In *2022 American Control Conference (ACC)*, pages 4422–4428, 2022.

[Steuer and Choo, 1983] Ralph E. Steuer and Eng-Ung Choo. An interactive weighted tchebycheff procedure for multiple objective programming. *Math. Program.*, 26(3):326–344, October 1983.

[Towers *et al.*, 2024] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[Vamplew *et al.*, 2011] Peter Vamplew, Richard Dazeley, Adam Berry, Rustam Issabekov, and Evan Dekker. Empirical evaluation methods for multiobjective reinforcement learning algorithms. *Mach. Learn.*, 84(1–2):51–80, July 2011.

[Van Moffaert *et al.*, 2013] Kristof Van Moffaert, Madalina M. Drugan, and Ann Nowé. Scalarized multi-objective reinforcement learning: Novel design techniques. In *2013 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*, pages 191–199, 2013.

[Veličković *et al.*, 2018] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

[Wang *et al.*, 2025] Jiaxing Wang, Yifeng Yu, Jiahan Song, Bin Cao, Jing Fan, and Ji Zhang. Rlhgnn: Reinforcement learning-driven heterogeneous graph neural network for next activity prediction in business processes, 2025.

[Xu *et al.*, 2020] Jie Xu, Yunsheng Tian, Pingchuan Ma, Daniela Rus, Shinjiro Sueda, and Wojciech Matusik. Prediction-guided multi-objective reinforcement learning for continuous robot control. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

[Yang *et al.*, 2019] Runzhe Yang, Xingyuan Sun, and Karthik Narasimhan. A generalized algorithm for multi-objective reinforcement learning and policy adaptation. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

[Zhu *et al.*, 2023] Baiting Zhu, Meihua Dang, and Aditya Grover. Scaling pareto-efficient decision making via offline multi-objective rl. In *International Conference on Learning Representations*, 2023.

Appendices

A Pareto Front

In multi-objective optimization, a solution is Pareto optimal if it cannot be improved in any objective without degrading at least one other objective. Let's consider a multi-objective maximization problem with N objectives, represented by a vector function:

$$\mathbf{F}(x) = (f_1(x), f_2(x), \ldots, f_N(x)),$$

where x is a solution in the decision space X.

A solution $x_1 \in X$ is said to **dominate** another solution $x_2 \in X$, denoted as $x_1 \succ x_2$, if and only if:

1. $f_i(x_1) \geq f_i(x_2)$ for all objectives $i = 1, \ldots, N$.
2. $f_j(x_1) > f_j(x_2)$ for at least one objective $j \in \{1, \ldots, N\}$.

A solution $x^* \in X$ is called **Pareto optimal** if there is no other solution $x \in X$ that dominates it. The set of the objective values of all Pareto-optimal solutions form the **Pareto Front**.

B Hypervolume Definition

The hypervolume of a set of solutions generated from a particular policy is defined as follows:

$$\mathrm{HV}(X, \mathbf{r}) = \lambda \left(\bigcup_{\mathbf{x} \in X} \left([x_0, r_0] \times \cdots \times [x_{N-1}, r_{N-1}] \right) \right), \tag{1}$$

where:

- $X \subset \mathbb{R}^N$ is a finite set of objective vectors (solutions in the objective space),
- $\mathbf{x} = (x_0, x_1, \ldots, x_{N-1}) \in X$ is an individual objective vector,
- $\mathbf{r} = (r_0, r_1, \ldots, r_{N-1}) \in \mathbb{R}^N$ is the reference point, chosen such that $x_i \geq r_i$ for all i (maximization case),
- N is the number of objectives,
- $\lambda(\cdot)$ denotes the Lebesgue measure (i.e. the volume) of the dominated region.

The reference point is standardized to be at the origin for all problems, since all objective functions are designed to be non-negative.

C Ordering Score Computation

Refer to Algorithm 1 for Ordering Score computation.

Algorithm 1 Ordering Score Computation

Require: Environment env with N objectives, policy π_θ, number of samples n_{samp}, steps n_{step} in each parameter sweep
Ensure: Ordering score $\mathcal{O} \in [0, 1]$

Generate set of preferences $\mathbf{W}$ by sweeping each objective i from 0 to 1 in n_{step} steps, sampling other components uniformly via random Dirichlet samples (repeat n_{samp} times).
Evaluate π_θ $\forall \mathbf{w} \in \mathbf{W}$ to obtain a list of objective vectors.
for each objective i and repetition j **do**
    Extract reward sequence $\mathbf{J}_i^{(j)}$ across n_{step} steps.
    if all values in $\mathbf{J}_i^{(j)}$ are equal **then**
        $s_{i,j} \leftarrow 1$
    else
        $s_{i,j} \leftarrow \frac{1}{2}\left(\text{Spearman}(\mathbf{J}_i^{(j)}, \text{sorted}(\mathbf{J}_i^{(j)})) + 1\right)$
    end if
end for
return $\mathcal{O} = \frac{1}{N \cdot n_{\text{samp}}} \sum_{i=1}^{N} \sum_{j=1}^{n_{\text{samp}}} s_{i,j}$

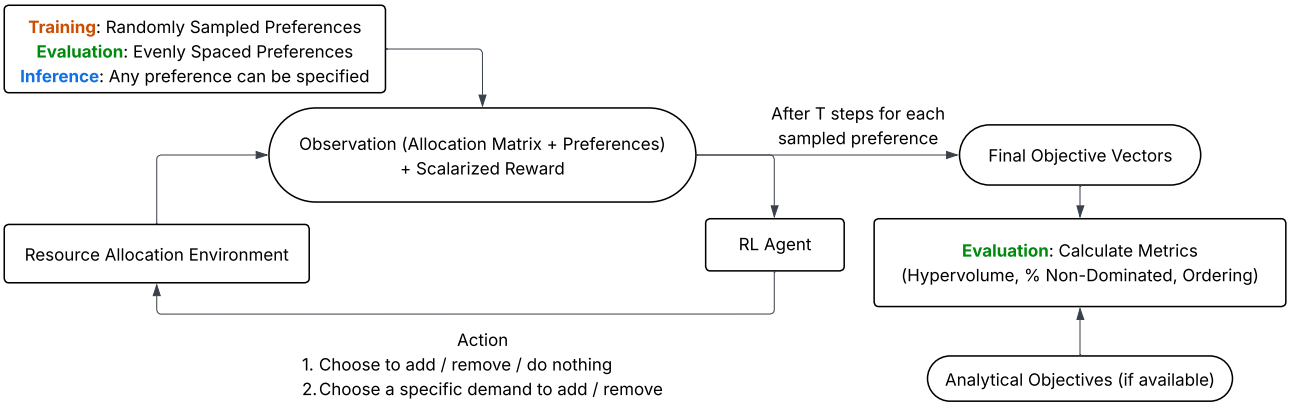

Figure 4: Training and evaluation pipeline using the CityPlannerEnv Gymnasium environment and Proximal Policy Optimization (PPO) agent from Stable Baselines3 [Raffin *et al.*, 2021].

D Training Methodology

D.1 PCPL-PPO Solution

Using the set of objective functions as our multi-objective reward function $\mathbf{J}$, we train a preference-conditioned policy using Proximal Policy Optimization (PPO) to convergence. We then employ the methods in Section 4 to evaluate the trained model. Figure 4 illustrates our training and evaluation pipeline. During training, preferences are sampled from a flat Dirichlet distribution for each rollout, and the PPO agent is trained to maximize the scalarized reward based on different preferences for each rollout, given the preference vector as input. Each rollout lasts for a total of T steps, which is part of the environment configuration set prior to training.

D.2 Normalization

Firstly, we normalized each component of the objective function to between 0 and 1, by taking the worst values of each component (i.e. the nadir point) to be at the origin (since objectives are defined to be always non-negative), and setting the best values (i.e. ideal point) to the current maximum reward obtained based on all prior rollouts. Even though using a moving ideal point may lead to some instability in training when discovering new ideal points, the agent eventually learns the optimal behavior over a large number of training steps.

D.3 Scalarization

Most existing RL-based scalarization methods use weighted sum scalarizations (i.e. Reward $= \mathbf{J} \cdot \mathbf{w}$) because of their simplicity. However, weighted-sum methods are ineffective in approximating non-convex Pareto Fronts. Therefore, we experimented with other scalarization methods, such as Tchebycheff scalarization [Steuer and Choo, 1983] and Penalty Boundary Intersection (PBI) [Deb and Jain, 2014]. We found that Smooth Tchebycheff Scalarization [Lin *et al.*, 2024] performed the most robustly in terms of hypervolume achieved when the agent encountered highly non-convex Pareto fronts. Additionally, the Smooth Tchebycheff Scalarization includes a smoothness hyperparameter that can be tuned for different problems with varying Pareto Front landscapes.

Unlike weighted sums, methods such as the Smooth Tchebycheff Scalarization require an ideal point to compute the scalarized reward, and this information is not readily available in an RL environment. Hence we use the moving ideal point [Van Moffaert *et al.*, 2013] introduced in Section D.2 as an estimate of the true ideal point.

D.4 Model Architecture

We used a Stable Baselines3 Proximal Policy Optimization (PPO) Agent with a Multi-Layer Perceptron (MLP) feature extractor (See Figure 5). This feature extractor can be easily replaced with alternate architectures such as Graph Neural Networks (Section 6), while maintaining the same feature vector dimension.

E Objective Function

See Table 3 for examples of objective functions defined in GraphAllocBench using CityPlannerEnv. Each component of the objective function $\mathbf{J}_i$ can depend on any production $\mathbf{P}_j$, and comprises elementary mathematical functions such as linear, quadratic, logarithmic, logistic, Gaussian, ceiling and floor functions. We can obtain complex objective functions based on the sum of elementary functions to design particular challenges (e.g. Gaussian with ceiling, sinusoidal increasing). All objectives

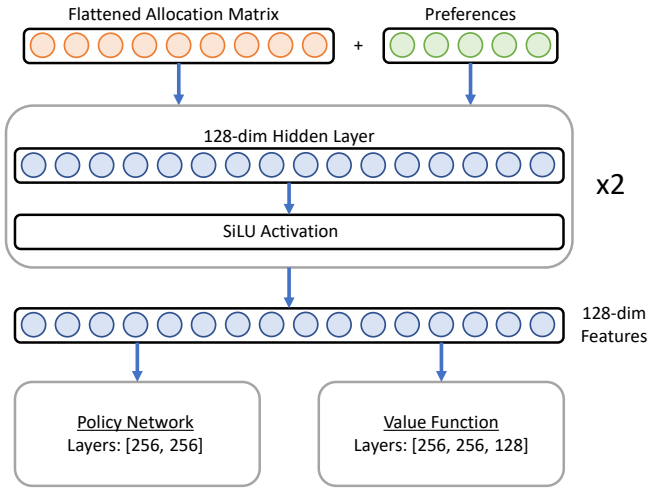

Figure 5: PPO architecture with a shared 2-layer MLP feature extractor (128 units, SiLU) for policy and value networks, implemented with Stable Baselines3.

Table 2: Evaluation Problem Set Description

Problems	$\|P\|$	N	**FC**	**Description**
0	2	2	Yes	Baseline simple logarithmic objective functions with smooth convex Pareto Fronts. The baseline problem uses a Fully-Connected (FC) dependency graph, where every demand depends on every resource.
1a-c	2	2	Yes	Difficult objective functions, including oscillatory behavior, stationary rewards, and spikes.
2a-c	5	2	Yes	More demands, which increases the action and observation spaces.
3a-b	5	5	Yes	5 objectives with sparse rewards, building on difficult objective functions from previous testcases.
4a-b	5	5	No	Varied dependencies and resources, instead of Fully-Connected (FC) dependency graphs.
5a-e	5	3-5	No	Testcases with dependencies, resources, and objectives sampled randomly. Extended horizon for allocating more available resources.
6a-c	100	5	No	Random testcases with simple convex functions similar to baseline, but with more complex graph structures with 100 demands and 100 resources. The ideal Pareto Front is not computed due to computational complexity.

*For more detailed problem definitions, refer to Appendix E.

are non-negative (i.e. $\mathbf{J}_i = \max(f(\mathbf{P}), 0)$) so that reward is always non-negative. For more detailed problem definitions on the full problem set, refer to https://anonymous.4open.science/r/GraphAllocBench.

F Results and Comparison with PD-MORL and Envelope Q-Learning

We ran PD-MORL [Basaklar *et al.*, 2023] (MO-DDQN-HER) and Envelope Q-Learning [Yang *et al.*, 2019] on all GraphAllocBench problems and compare against our PPO-based PCPL (PCPL-PPO) in Table 4. We conducted a per-problem random hyperparameter search over learning rate, discount factor, batch size, soft-update rate, replay buffer size, exploration schedule, weight batch size, and training step budget. The best configuration found per problem was then used to train 5 independent seeds at up to 1M steps.

The results illustrate how GraphAllocBench differentiates algorithm behavior across diverse problem settings. PCPL-PPO achieves higher normalized hypervolume on 10 of 16 problems and leads on PNDS and ordering score on the majority of problems. Envelope Q-Learning leads on hypervolume for Problems 2b and 5d, and achieves the highest PNDS and ordering score on Problem 2b, but generally underperforms PCPL-PPO and PD-MORL across the broader problem set, particularly on problems with sparse rewards (Problem 1c, 3a–3b) and high-dimensional observations (5b–5c). PD-MORL remains competitive on HV for Problems 1b, 1c, 5a, and 5b, but on 5a and 5b its narrow HV margin does not extend to PNDS or ordering score, suggesting the benchmark's supplementary metrics expose preference-alignment failures that hypervolume alone would miss. On Problems 1b and 1c, which feature sharp and oscillatory objective functions, PD-MORL leads on all three metrics after hyperparameter tuning, though HV on 1c remains highly variable (0.312 ± 0.302), reflecting seed-level instability under oscillatory rewards.

Table 3: Objective functions for selected problems (zero lower bound assumed for all functions).

Problem	Objective Function $\mathbf{J}$
0	$\mathbf{J}_0 = 10\log(\mathbf{P}_0 + \epsilon + 1)$ $\mathbf{J}_1 = 10\log(\mathbf{P}_1 + \epsilon + 1)$
1a	$\mathbf{J}_0 = \min(0.6\mathbf{P}_0^2, 18) + \max(5\exp(-0.9(\mathbf{P}_1 - 7)^2, 0.1))$ $\mathbf{J}_1 = -1.5\mathbf{P}_0^2 + 0.8\mathbf{P}_0 + 12 + 5\exp(-0.1(\mathbf{P}_1 - 3)^2) + 0.2\mathbf{P}_1\log(2\mathbf{P}_1 + \epsilon + 1)$
1b	$\mathbf{J}_0 = 0.5\mathbf{P}_0 + 3 + 3\sin(3\mathbf{P}_0) - \log(1.5\mathbf{P}_1 + \epsilon + 2) + 2.5\exp(-1.2(\mathbf{P}_1 - 7)^2)$ $\mathbf{J}_1 = -1.5\log(5.5\mathbf{P}_0 + \epsilon + 1) + 9\exp(-0.5(\mathbf{P}_0 - 2)^2) + 5 + (0.5\mathbf{P}_1 + 1.2)\sin(0.9\mathbf{P}_1 + 0.6)$
1c	$\mathbf{J}_0 = -\mathbf{P}_0 + \frac{20}{1+\exp(\mathbf{P}_1 - 3)}$ $\mathbf{J}_1 = \frac{1}{1+\exp(\mathbf{P}_0 - 6)} + 5 - \frac{15}{1+\exp(0.7(\mathbf{P}_1 - 5))}$

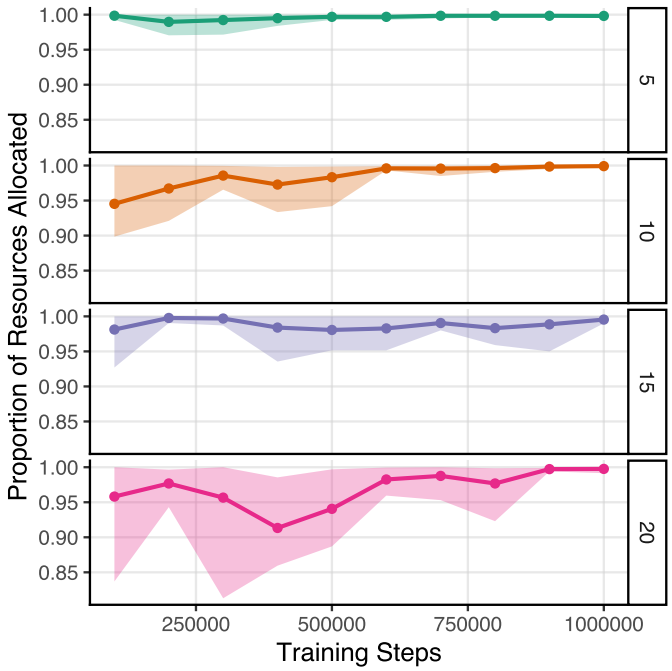

Figure 6: Performance over different number of objectives (5, 10, 15, 20) for a simple increasing objective function, across 5 random seeds. The points show the mean performance, and the shaded regions show the range over 5 seeds.

The benchmark's diversity also reveals where all methods struggle most: on problems with sparse rewards, complex dependency structures, or unbalanced objectives (Problems 2c, 3a, 3b, 4b), both PD-MORL and Envelope Q-Learning see substantially degraded performance, while PCPL-PPO maintains a larger lead. These results demonstrate that GraphAllocBench's varied problem set and complementary metrics can expose distinct failure modes in preference-conditioned policy learning that simpler benchmarks would obscure.

G Effect of Increasing Number of Objectives

We study the effect of increasing the number of objectives (Figure 6) by modeling each objective as dependent on a unique production (i.e. $N = |\mathbf{P}|$) and using objective functions that monotonically increase in each production. As the true Pareto Front is computationally intensive to calculate for high-dimensional objectives, we use the ratio of the quantity of allocated resources to total resources available to estimate the proximity of the predicted solutions to the true Pareto Front; intuitively, if objective functions are always monotonically increasing, it is always better to use more resources to get more reward, regardless of preference. Hence, the Pareto Front consists of outcomes that use all available resources in the environment. In Figure 6, results suggest that the agent maintains relatively stable performance even as the objective space dimensionality quadruples for simple objective functions, given sufficient model capacity and number of training steps. Additionally, a relatively high 20-objective ordering score (0.89) can still be reached at 1M steps. This restricted setup enables us to conclude that increasing

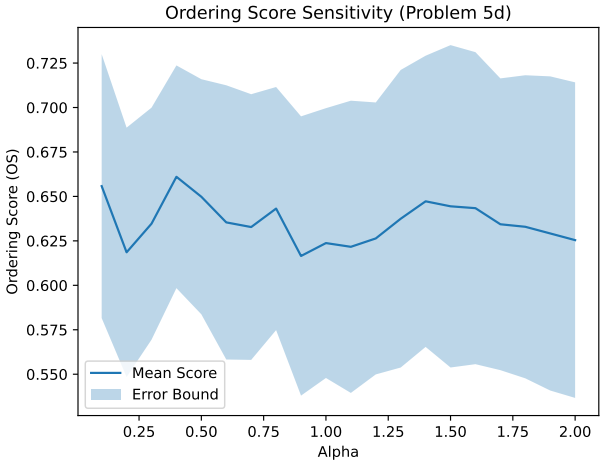

Figure 7: Order Sensitivity (Mean + Standard Deviation) across Dirichlet Distribution α parameter (Problem 5d)

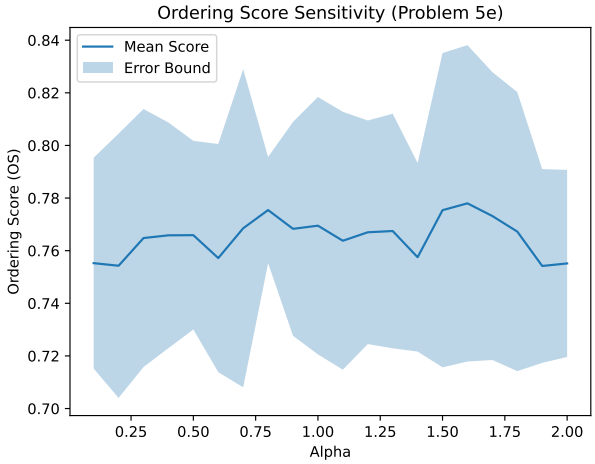

Figure 8: Order Sensitivity (Mean + Standard Deviation) across Dirichlet Distribution α parameter (Problem 5e)

the number of objectives (and productions) with a simple objective function and dependency setup may not necessarily lead to a significant reduction in overall performance, as measured by the proportion of allocated resources as an estimate of the closeness to the ideal solution. This result concurs with the findings of similar tests [Ishibuchi *et al.*, 2016] conducted using evolutionary methods for multi-objective optimization on benchmarks such as DTLZ.

H Ordering Score Sensitivity

To test the sensitivity of the ordering score, we calculated the ordering score for Problems 5d and 5e across 5 random seeds with different sampling patterns by varying values of α for the Dirichlet Distribution sampling in Algorithm 1. For simple problems (e.g. Problem 0), the ordering score remained relatively constant. For more complex problems, ordering scores may fluctuate with different sampling methods. To investigate the sensitivity of Ordering Score with respect to sampling, Problem 5d and 5e were selected for their higher number of objectives (5), varied dependencies and low ordering score. The variation in Ordering Score is shown in Figures 7 and 8.

In Figures 7 and 8, although there are fluctuations in ordering score due to the variations in policies obtained, the mean ordering score remains relatively stable over different alpha parameters for the Dirichlet distribution, indicating resilience over different sampling methods.

Table 4: Comparison of PD-MORL, Envelope Q-Learning, and PCPL-PPO across all benchmark problems (mean $\pm$ std, n=5 seeds). PD-MORL and Envelope results use per-problem hyperparameters selected via random search. Bold indicates the best result per metric per problem.

Problem	Method	Norm. HV ↑	PNDS ↑	Ordering ↑
0	PD-MORL	0.986 ± 0.008	0.771 ± 0.196	$\mathbf{1.000 \pm 0.000}$
	Envelope	0.916 ± 0.023	0.352 ± 0.234	0.944 ± 0.037
	PCPL-PPO	$\mathbf{0.997 \pm 0.007}$	$\mathbf{1.000 \pm 0.000}$	$\mathbf{1.000 \pm 0.000}$
1a	PD-MORL	0.646 ± 0.064	0.533 ± 0.174	$\mathbf{0.952 \pm 0.043}$
	Envelope	0.491 ± 0.172	0.533 ± 0.224	0.934 ± 0.037
	PCPL-PPO	$\mathbf{0.724 \pm 0.049}$	$\mathbf{0.615 \pm 0.169}$	0.947 ± 0.008
1b	PD-MORL	$\mathbf{0.645 \pm 0.039}$	$\mathbf{0.933 \pm 0.071}$	$\mathbf{0.995 \pm 0.011}$
	Envelope	0.444 ± 0.226	0.552 ± 0.223	0.752 ± 0.191
	PCPL-PPO	0.632 ± 0.067	0.738 ± 0.151	0.986 ± 0.008
1c	PD-MORL	$\mathbf{0.312 \pm 0.302}$	$\mathbf{0.438 \pm 0.285}$	$\mathbf{0.975 \pm 0.050}$
	Envelope	0.183 ± 0.201	0.276 ± 0.363	0.896 ± 0.088
	PCPL-PPO	0.230 ± 0.188	0.215 ± 0.090	0.945 ± 0.089
2a	PD-MORL	0.238 ± 0.188	0.133 ± 0.070	$\mathbf{1.000 \pm 0.000}$
	Envelope	0.224 ± 0.203	0.533 ± 0.347	0.680 ± 0.086
	PCPL-PPO	$\mathbf{0.989 \pm 0.004}$	$\mathbf{1.000 \pm 0.000}$	$\mathbf{1.000 \pm 0.000}$
2b	PD-MORL	0.560 ± 0.015	0.648 ± 0.246	0.968 ± 0.026
	Envelope	$\mathbf{0.607 \pm 0.134}$	$\mathbf{0.771 \pm 0.152}$	$\mathbf{0.968 \pm 0.011}$
	PCPL-PPO	0.605 ± 0.163	0.554 ± 0.277	0.896 ± 0.057
2c	PD-MORL	0.200 ± 0.042	0.514 ± 0.288	0.941 ± 0.059
	Envelope	0.174 ± 0.078	0.162 ± 0.123	0.837 ± 0.115
	PCPL-PPO	$\mathbf{0.680 \pm 0.084}$	$\mathbf{0.769 \pm 0.138}$	$\mathbf{0.987 \pm 0.016}$
3a	PD-MORL	0.505 ± 0.104	0.014 ± 0.014	0.860 ± 0.029
	Envelope	0.288 ± 0.221	0.031 ± 0.041	0.732 ± 0.035
	PCPL-PPO	$\mathbf{0.827 \pm 0.047}$	$\mathbf{0.268 \pm 0.020}$	$\mathbf{0.917 \pm 0.043}$
3b	PD-MORL	0.053 ± 0.098	0.006 ± 0.005	0.797 ± 0.068
	Envelope	0.190 ± 0.106	0.011 ± 0.008	0.774 ± 0.060
	PCPL-PPO	$\mathbf{0.301 \pm 0.138}$	$\mathbf{0.014 \pm 0.010}$	$\mathbf{0.866 \pm 0.031}$
4a	PD-MORL	0.695 ± 0.157	$\mathbf{0.026 \pm 0.046}$	0.757 ± 0.084
	Envelope	0.569 ± 0.145	0.005 ± 0.006	0.674 ± 0.079
	PCPL-PPO	$\mathbf{0.829 \pm 0.113}$	0.017 ± 0.004	$\mathbf{0.854 \pm 0.064}$
4b	PD-MORL	0.417 ± 0.238	0.093 ± 0.078	0.644 ± 0.143
	Envelope	0.185 ± 0.123	0.327 ± 0.148	0.715 ± 0.095
	PCPL-PPO	$\mathbf{0.802 \pm 0.100}$	$\mathbf{0.718 \pm 0.132}$	$\mathbf{0.963 \pm 0.018}$
5a	PD-MORL	$\mathbf{0.595 \pm 0.006}$	0.153 ± 0.233	0.849 ± 0.104
	Envelope	0.470 ± 0.029	0.050 ± 0.027	0.875 ± 0.024
	PCPL-PPO	0.588 ± 0.025	$\mathbf{0.262 \pm 0.071}$	$\mathbf{0.979 \pm 0.008}$
5b	PD-MORL	$\mathbf{0.957 \pm 0.003}$	0.095 ± 0.078	0.778 ± 0.109
	Envelope	0.271 ± 0.267	0.036 ± 0.030	0.548 ± 0.039
	PCPL-PPO	0.940 ± 0.035	$\mathbf{0.121 \pm 0.053}$	$\mathbf{0.943 \pm 0.021}$
5c	PD-MORL	0.880 ± 0.075	0.059 ± 0.035	0.744 ± 0.101
	Envelope	0.450 ± 0.142	0.081 ± 0.046	0.485 ± 0.106
	PCPL-PPO	$\mathbf{0.905 \pm 0.038}$	$\mathbf{0.648 \pm 0.399}$	$\mathbf{0.916 \pm 0.047}$
5d	PD-MORL	0.678 ± 0.136	$\mathbf{0.033 \pm 0.018}$	$\mathbf{0.665 \pm 0.081}$
	Envelope	$\mathbf{0.730 \pm 0.175}$	0.021 ± 0.018	0.548 ± 0.076
	PCPL-PPO	0.699 ± 0.103	0.027 ± 0.016	0.621 ± 0.066
5e	PD-MORL	0.633 ± 0.022	$\mathbf{0.352 \pm 0.172}$	0.595 ± 0.071
	Envelope	0.574 ± 0.095	0.150 ± 0.067	0.748 ± 0.040
	PCPL-PPO	$\mathbf{0.721 \pm 0.066}$	0.288 ± 0.076	$\mathbf{0.768 \pm 0.099}$